
Software Requirements Specification

for

Global Student Connect

An AI-Powered Student Networking System

Version 1.2 approved

**Prepared by: Tazrian Chowdhury 2414248042
Fahima Afrin Anuva 2321049042
Md. Tushar Khan 2232344042**

North South University

August 2026

Table of Contents

Table of Contents	2
Revision History	4
1. Introduction	5
1.1 Purpose	5
1.2 Document Conventions	5
1.3 Intended Audience and Reading Suggestions	5
1.4 Product Scope	5
1.5 References	5
2. Overall Description	7
2.1 Product Perspective	7
2.2 Product Functions	7
2.3 User Classes and Characteristics	7
2.4 Operating Environment	7
2.5 Design and Implementation Constraints	7
2.6 User Documentation	8
2.7 Assumptions and Dependencies	8
3. External Interface Requirements	10
3.1 User Interfaces	10
3.2 Hardware Interfaces	10
3.3 Software Interfaces	10
3.4 Communications Interfaces	10
4. System Features	11
4.1 Student Verification (Verification Agent)	11
4.1.1 Description and Priority	11
4.1.2 Stimulus/Response Sequences	11
4.1.3 Functional Requirements	11
4.2 Profile Management	12
4.2.1 Description and Priority	12
4.2.2 Stimulus/Response Sequences	12
4.2.3 Functional Requirements	12
4.3 AI-Based Matching (Matching Agent)	12
4.3.1 Description and Priority	12
4.3.2 Stimulus/Response Sequences	12
4.3.3 Functional Requirements	13
4.4 Moments Feed	13
4.4.1 Description and Priority	13
4.4.2 Stimulus/Response Sequences	13
4.4.3 Functional Requirements	14
4.5 Messaging System	14
4.5.1 Description and Priority	14
4.5.2 Stimulus/Response Sequences	14
4.5.3 Functional Requirements	14
4.6 AI Translation (Translation Agent)	14

4.6.1 Description and Priority	14
4.6.2 Stimulus/Response Sequences	14
4.6.3 Functional Requirements	15
4.7 Notifications	16
4.7.1 Description and Priority	16
4.7.2 Stimulus/Response Sequences	16
4.7.3 Functional Requirements	16
4.8 Missions & Badges	16
4.8.1 Description and Priority	16
4.8.2 Stimulus/Response Sequences	16
4.8.3 Functional Requirements	16
4.9 AI Safety & Moderation (Safety Agent)	16
4.9.1 Description and Priority	17
4.9.2 Stimulus/Response Sequences	17
4.9.3 Functional Requirements	17
4.10 Progress Nudges (Nudge Agent)	17
4.10.1 Description and Priority	17
4.10.2 Stimulus/Response Sequences	18
4.10.3 Functional Requirements	18
4.11 Platform Assistant (Platform Assistant Agent)	18
4.11.1 Description and Priority	18
4.11.2 Stimulus/Response Sequences	18
4.11.3 Functional Requirements	18
5. Other Nonfunctional Requirements	20
5.1 Performance Requirements	20
5.2 Safety Requirements	20
5.3 Security Requirements	20
5.4 Software Quality Attributes	20
5.5 Business Rules	21
6. Other Requirements	22
Appendix A: Glossary	23
Appendix B: Analysis Models	24
B.1 Use Case Diagram	24
B.2 Detailed Use Case Description: Find a Friend	25
B.3 Sequence Diagrams	26
B.4 Communication Diagram	28
B.5 Activity Diagram: Find a Friend	28
B.6 Class Diagram	31
Appendix C: To Be Determined List	32

Revision History

Name	Date	Reason For Changes	Version
Team	July 2026	Initial draft	1.0
Team	July 2026	Tiered badges; Verification/Matching agents upgraded to real LLM calls; Badges & Missions nudge notifications added	1.1
Team	August 2026	Restructured AI layer into six agents (Verification, Matching, Translation, Safety, Nudge, Platform Assistant), all backed by Google Gemini across two API keys; added the ai_agents Django app (SafetyFlag, NudgeLog, AssistantThread, AssistantMessage); added User.suspended_until and MessageTranslation.is_fallback; defined graceful-degradation behavior for every agent; resolved prior TBDs on unrecognized-domain handling, real-time delivery mechanism, and account-suspension model; added Appendix B (Analysis Models)	1.2

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for Global Student Connect, Version 1.2. Global Student Connect is a web-based social networking platform designed to help international university students discover peers with shared interests, communicate across language and university boundaries, and build community through AI-assisted features. This document covers the complete system: student verification, profile management, matching, the moments feed, messaging, translation, notifications, the missions/badges gamification layer, progress nudges, safety moderation, and the Platform Assistant.

1.2 Document Conventions

Functional requirements are labeled REQ-<feature>-<number> (e.g., REQ-MATCH-1) and grouped under the system feature they belong to. Each requirement is assumed to inherit the priority (High/Medium/Low) assigned to its parent feature unless otherwise stated. Database entity names (e.g., User, Connection, Block) are written in title case throughout this document to match the underlying data model. The word "shall" indicates a mandatory requirement; "should" indicates a recommended but non-mandatory behavior.

1.3 Intended Audience and Reading Suggestions

This document is intended for the three student developers building the system, their course instructor/evaluator, and any future contributor who needs to understand system behavior without reading the source code. Developers should read Sections 2 through 5 in order before implementation. Evaluators primarily interested in scope and feature completeness may focus on Sections 1, 2, and 4. Section 6 and the Appendices provide supplementary reference material and may be read as needed.

1.4 Product Scope

Global Student Connect aims to reduce the isolation international students often feel by making it easy to find compatible peers, join interest-based groups, and communicate despite language differences. The product's core value is threefold: (1) verified, university-aware identity so students trust who they are talking to, (2) AI-assisted matching, translation, safety moderation, and platform guidance that lower the effort of building new connections, and (3) a lightweight gamification layer that encourages continued engagement through tiered badges and personalized nudges.

This SRS specifies six AI-driven agents — Verification, Matching, Translation, Safety, Nudge, and Platform Assistant — all implemented as genuine calls to the Google Gemini API (Section 2.5). As of this revision, the Safety Agent is implemented and in active use; the Matching Agent's automated find-and-request flow is implemented; the Nudge Agent is the next feature in the implementation order; the Translation Agent is in progress. Every agent is specified here at its target end-state behavior, including its fallback behavior on API failure, regardless of implementation status at the time of writing.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications (template basis for this document)
- Global Student Connect — Full Project Brief (internal project document: database schema, Block/End Chat behavior, university verification mechanism, Django app structure)
- Global Student Connect — Association Reference and Class Diagram Reference (internal project documents: full class/attribute/relationship listings, including the ai_agents app)
- Django Software Foundation, Django Documentation, <https://docs.djangoproject.com/>

- Google Identity Platform, Google OAuth 2.0 Documentation, <https://developers.google.com/identity>
- Google AI for Developers, Gemini API Documentation, <https://ai.google.dev/>

2. Overall Description

2.1 Product Perspective

Global Student Connect is a new, self-contained product. A complete front-end prototype (HTML/CSS/JavaScript with a localStorage data layer) already exists and defines the visual design system and page flow; this SRS specifies the requirements for the production system, rebuilt on a Django (Python) backend with a MySQL database. The system is not a component of a larger existing product and has no legacy system it replaces. It authenticates users via Google OAuth rather than a self-managed credential store.

2.2 Product Functions

At a high level, the system shall let a user do the following:

- Verify their status as a university student and have their university identified from their email domain
- Create and edit a profile: display name, country, languages, interests
- Be matched one-to-one with the single best-fitting candidate on request, or join interest-based groups instantly
- Share text posts (“moments”), and like/comment on others' posts
- Exchange one-to-one and group messages, with on-demand translation
- Receive notifications for matches, messages, mission progress, and system events
- Earn tiered badges by completing missions tied to platform engagement, and receive personalized progress nudges
- Get contextual help navigating the platform from the built-in Platform Assistant
- Report inappropriate content or users for moderator review, with new content additionally screened automatically before it becomes visible
- Block another user, removing all mutual visibility across the platform, or simply end an individual chat without that broader effect

2.3 User Classes and Characteristics

Two user classes are defined:

- Student (Verified) — the primary user class. A person with a valid, verifiable university email address. Uses nearly all product functions: profile, matching, feed, messaging, notifications, missions/badges, the Platform Assistant, and reporting. No special technical expertise is assumed.
- Administrator/Moderator — a project team member (or, in production, platform staff) who uses the built-in Django admin interface to correct auto-generated University names and review submitted Reports and SafetyFlags. This class does not use the student-facing pages at all.

2.4 Operating Environment

The system is a web application accessed through modern desktop and mobile browsers (Chrome, Firefox, Safari, Edge, current and prior major version). The backend runs on Django (Python 3) with a MySQL relational database. No native mobile application is in scope for this version.

2.5 Design and Implementation Constraints

- The database schema now spans 7 Django apps — accounts, social, matching, chat, engagement, moderation, and ai_agents — specified in the project brief and the Association/Class Diagram Reference documents; this SRS's data-related requirements must remain consistent with them.
- Authentication is via Google OAuth only; the system does not implement its own password storage.
- Six AI-driven agents — Verification, Matching, Translation, Safety, Nudge, and Platform Assistant — shall be implemented as genuine calls to the Google Gemini API, split across two API keys by function: a primary key (Matching, Safety, Nudge) and a secondary key (Verification, Translation, Platform Assistant). Badge/mission threshold-tracking and badge-awarding logic itself shall remain fully deterministic (never AI-driven), to avoid incorrectly awarding a badge; the Nudge Agent's role is limited to generating personalized nudge text from a user's existing badge/mission progress data and to rate-limiting itself, and never influences whether a badge is actually awarded.
- The Verification Agent consists of two independent sub-behaviors on two different triggers: a deterministic, rule-based email-domain-suffix check that sets User.is_verified (no LLM involved), and a separate Gemini call that fires only once per new domain (on the first student ever from that domain) to generate a candidate real-world institution name for the University record, in place of a naive domain-derived guess. Both are described fully in Section 4.1.
- The Safety Agent's moderation check is synchronous: a chat message, post, or comment does not become visible to anyone until the Gemini safety check completes or its fallback triggers (Section 4.9).
- Every Gemini-backed agent shall degrade gracefully rather than block core functionality on API failure, timeout, or quota exhaustion, using one of the following agent-specific fallback strategies: the Verification Agent (name-generation call) falls back to the deterministic domain.split('.')[0] guess from the original project brief; the Matching Agent falls back to a deterministic in-code compatibility-scoring algorithm, invisibly to the user; the Safety Agent fails open — the content is still delivered/posted, with a brief non-blocking on-screen notice shown to the sender; the Translation Agent stores the original, untranslated text as a placeholder and flags it for retry on next view; the Nudge and Platform Assistant agents degrade with a non-blocking on-screen notice, consistent with the Safety Agent's approach. In every case, the platform's core functionality continues to work even when Gemini is unavailable.
- A user may hold at most 5 interests, enforced in application logic rather than as a database constraint.
- Gender is collected and stored for use in same-gender-aware matching only; it is never displayed as a public preference.
- The Platform Assistant is scoped strictly to platform help and navigation guidance (explaining how features work, where to find things); it is explicitly not a general-purpose writing or content-generation tool and shall not be used to draft essays or other content on the user's behalf.

2.6 User Documentation

An in-app onboarding flow (profile setup wizard) substitutes for a formal user manual. No separate printed or PDF user guide is planned for Version 1.2. The Platform Assistant (Section 4.11) additionally provides in-context help for navigating the product. Tooltips/help text on the reporting and blocking flows should be sufficient for a user to understand the consequences of each action without external documentation.

2.7 Assumptions and Dependencies

- It is assumed that Google OAuth reliably returns the hd (hosted domain) field for institutional Google Workspace accounts, which the verification step depends on.

- It is assumed that university email domains follow recognizable academic suffix patterns; a domain that matches no known pattern blocks sign-in with a warning message (REQ-VER-3), rather than being an open question.
- The project depends on the Google Gemini API remaining available and within free-tier or education-tier usage limits, across both API keys described in Section 2.5. Every Gemini-backed agent has a defined fallback for when this dependency is not met (Section 2.5), so a Gemini outage degrades individual features rather than the platform as a whole.
- It is assumed that all three team members will have working Django/MySQL environments set up as described in the project brief's Phase 0.

3. External Interface Requirements

3.1 User Interfaces

The system is delivered as 9 primary screens, consistent with the existing prototype: Sign-in, Profile Setup, Home (Feed), Connect (Matching), Chat Inbox, Direct Conversation, Group Conversation, Profile, and Notifications, plus the Platform Assistant panel. The established design system shall be followed: light background (#f8f7ff), dark navy sidebar (#1e1560), purple primary accent (#3d2fd9), gold accent (#f5a623), white cards with #e8e6ff borders, Plus Jakarta Sans typography, and Phosphor Icons. HTML structure and CSS styling are maintained in separate files; no inline styles are used. A persistent sidebar provides navigation across all authenticated pages.

3.2 Hardware Interfaces

No dedicated hardware interfaces are required. The system runs entirely over standard consumer hardware (desktop, laptop, tablet, or smartphone) with a network connection; no direct hardware-level integration (cameras, sensors, peripherals) is in scope.

3.3 Software Interfaces

- Google OAuth 2.0 — used for authentication and initial retrieval of the user's email/domain for verification.
- MySQL — the relational database storing all persistent application data, per the schema in the project brief and the Association/Class Diagram Reference documents.
- Django (Python) — the web application framework serving all views, forms, and the admin interface.
- Google Gemini API, primary key — powers the Matching Agent (candidate compatibility scoring and single-best-candidate selection, with a deterministic in-code fallback), the Safety Agent (synchronous content moderation for chat messages, posts, and comments, with fail-open fallback), and the Nudge Agent (badge/mission progress nudge text generation).
- Google Gemini API, secondary key — powers the Verification Agent (university name generation, with deterministic fallback), the Translation Agent (message translation, with stored-placeholder-and-retry fallback), and the Platform Assistant Agent (platform help/navigation guidance, with non-blocking fallback).

3.4 Communications Interfaces

All client-server communication occurs over HTTPS, including calls made to the Google Gemini API. The system does not send outbound email or SMS in Version 1.2; all notifications are in-app only. Message delivery in Chat is implemented via client-side polling rather than a persistent WebSocket connection.

4. System Features

Features are organized around the product's six AI-assisted agents plus the core social/utility functions. Block and End Chat are two distinct actions that cut across Matching and Messaging respectively, and are specified within those two features rather than as standalone items.

4.1 Student Verification (Verification Agent)

4.1.1 Description and Priority

Confirms that a person signing in is a genuine university student via a deterministic domain-suffix check, and separately uses the Verification Agent to generate a human-readable university name for a newly seen domain, without requiring any manual data entry. Priority: High.

4.1.2 Stimulus/Response Sequences

- User signs in with Google → system reads the account's email domain → system checks the domain suffix against known academic patterns → on match, `is_verified` is set true; on no match, sign-in is blocked and the user is shown a warning that their email could not be verified as an academic address.
- System checks the full email domain against stored University records → the first student ever from a new domain triggers the Verification Agent, which calls the Google Gemini API (secondary key) to generate a candidate real institution name from the domain (e.g., “buet.ac.bd” → “Bangladesh University of Engineering and Technology”) and creates the University row with that name; every subsequent student from the same domain is instantly associated with the existing row, with no further Agent call.
- If the Gemini call fails, the system falls back to the deterministic `domain.split('.')[0]` guess (e.g., “buet”) so that account creation is never blocked by the Agent being unavailable.
- Administrator edits an auto-generated University name in Django admin, if the guess needs correction → the corrected name applies retroactively to all students from that domain, logged automatically.

4.1.3 Functional Requirements

REQ-VER-1: The system shall extract the email domain returned by Google OAuth at sign-in.

REQ-VER-2: The system shall mark a User as verified (`is_verified = true`) if the email domain suffix matches a recognized academic pattern (e.g., `.edu`, `.ac.bd`, `.ac.uk`, `.edu.au`).

REQ-VER-3: If the email domain suffix matches no recognized academic pattern, the system shall block sign-in and display a message informing the user that their email could not be verified as an academic address.

REQ-VER-4: The Verification Agent shall call the Google Gemini API (secondary key) to generate a candidate real-world institution name for a previously unseen academic domain, and the system shall automatically create a new University record using that name.

REQ-VER-5: If the Gemini call in REQ-VER-4 fails, times out, or is unavailable, the system shall fall back to a deterministic name guess derived from the domain string, and shall proceed with account creation without blocking on the Agent.

REQ-VER-6: The system shall associate every subsequent user from a known domain with the existing University record without requiring manual input.

REQ-VER-7: The system shall expose University records in the Django admin interface with searchable domain and name fields, and log all edits automatically, allowing an administrator to correct a generated name if needed.

4.2 Profile Management

4.2.1 Description and Priority

Allows a verified student to create and maintain the personal information used across matching, the feed, and messaging. Priority: High.

4.2.2 Stimulus/Response Sequences

- New verified user is redirected to the Profile Setup wizard → enters display name, country, languages, and up to 5 interests → completes setup and lands on the Home feed.
- Existing user navigates to their Profile page → edits any field → saves → changes apply immediately across the platform.

4.2.3 Functional Requirements

REQ-PROF-1: The system shall require display name, country, primary language, and at least one interest before marking `profile_setup_complete` as true.

REQ-PROF-2: The system shall allow a user to select at most 5 interests from a fixed set of 12 (Music, Travel, Reading, Photography, Art & Design, Cricket, Football, Badminton, Cooking, Coding, Movies & TV, Gaming).

REQ-PROF-3: The system shall collect gender for internal matching purposes only and shall never display it as a public-facing preference.

REQ-PROF-4: The system shall allow a user to set a secondary language and an auto-translate preference (on/off) and target language.

REQ-PROF-5: The system shall allow a user to edit any profile field after initial setup, with changes taking effect immediately.

4.3 AI-Based Matching (Matching Agent)

4.3.1 Description and Priority

Uses the Matching Agent to automatically find and request the single best-fitting candidate for a student on demand, without presenting a ranked list to choose from, and to support instant group joining. Also governs Block, which removes a user from another's matching pool permanently. Priority: High.

4.3.2 Stimulus/Response Sequences

- User opens Connect and is presented with two options: Find Friend and Join Group.
- User selects Find Friend → the Matching Agent calls the Google Gemini API (primary key) to score the current candidate pool and select the single highest-scoring candidate → the system automatically creates and sends a `MatchRequest` (status = pending) to that candidate on the requester's behalf → the requester sees a waiting screen.
- If the Gemini call fails, the system falls back to a deterministic in-code compatibility-scoring algorithm and proceeds identically from the requester's point of view — no error or fallback notice is shown for Matching.
- While a request the user sent is still pending, the system prevents them from initiating another Find Friend request until the current one is accepted, declined, or cancelled.
- The recipient sees a handshake-icon indicator appear on their own Connect page and receives a Notification. Opening it shows a modal listing all pending incoming requests, each reviewable individually as accept or decline.

- On accept: a Connection and its Conversation are created; the requester's waiting screen resolves to reveal who was matched, along with the stored compatibility score shown as a multicolored bar with its percentage value (e.g., 100%, 50%).
- On decline (or requester cancellation): the MatchRequest is closed with no Connection created; once no further requests remain queued for the recipient, they see an “all caught up” state.
- User joins a Group directly from a group listing → membership is instant, with no request/accept step.
- User blocks another user → the Matching Agent excludes the blocked user from all future candidate pools in both directions, and any active Connection between them is ended.

4.3.3 Functional Requirements

REQ-MATCH-1: The Matching Agent shall call the Google Gemini API (primary key) to compute a compatibility score for each available candidate in the requester's pool and select the single highest-scoring candidate.

REQ-MATCH-2: If the Gemini call in REQ-MATCH-1 fails, times out, or is unavailable, the system shall fall back to a deterministic in-code compatibility-scoring algorithm, transparently to the user, with no error or fallback notice shown.

REQ-MATCH-3: The system shall automatically create and send a MatchRequest on the requester's behalf to the candidate selected in REQ-MATCH-1/2, with status pending, accepted, declined, or cancelled.

REQ-MATCH-4: The system shall prevent a user from initiating a new Find Friend request while they already have a MatchRequest they sent that is still in pending status.

REQ-MATCH-5: The system shall create exactly one Connection when a MatchRequest is accepted, and shall create its associated Conversation at the same time.

REQ-MATCH-6: The system shall allow instant Group joining without a request/accept step, distinct from one-to-one matching.

REQ-MATCH-7: On acceptance, the system shall display the stored compatibility_score to the requester as a graphical, multicolored bar alongside its percentage value.

REQ-MATCH-8: The system shall allow a user to block another user. Blocking shall end any active Connection between them (ended_reason = block) and shall permanently exclude the blocked user from future candidate pools in both directions.

REQ-MATCH-9: The system shall enforce that a Block relationship is unique per (blocker, blocked) pair — a user cannot block the same person twice.

REQ-MATCH-10: The system shall enforce Block-based content visibility symmetrically: if User A blocks User B, neither party shall see the other's posts, comments, or match candidacy, even though the Block record itself is directional.

4.4 Moments Feed

4.4.1 Description and Priority

Lets students share short text updates and engage with others' updates via likes and comments. Priority: Medium.

4.4.2 Stimulus/Response Sequences

- User composes a text post from the Home feed → post appears in the feed for other students (subject to Block exclusions and Safety Agent screening, Section 4.9).
- User likes or comments on a visible post → the count/thread updates for all viewers, with new comments subject to the same Safety Agent screening as posts and messages.

4.4.3 Functional Requirements

REQ-FEED-1: The system shall allow a verified student to create a text post.

REQ-FEED-2: The system shall allow any student to like a post at most once, and to add one or more comments to a post.

REQ-FEED-3: The system shall exclude posts and comments authored by a blocked user from the viewing user's feed, per REQ-MATCH-10.

REQ-FEED-4: Ending a chat (see REQ-MSG-4) shall have no effect on feed visibility — two users who ended a chat shall continue to see each other's posts and comments normally.

4.5 Messaging System

4.5.1 Description and Priority

Provides one-to-one and group text messaging over established Connections and Groups, including the End Chat action. Priority: High.

4.5.2 Stimulus/Response Sequences

- User opens Chat inbox → selects an active conversation → sends and receives messages, delivered via client-side polling.
- User selects End Chat on a direct conversation → the Connection is marked ended → the conversation becomes inaccessible for future messaging, but the two users' feed content remains mutually visible.

4.5.3 Functional Requirements

REQ-MSG-1: The system shall support one-to-one messaging over an active Connection's Conversation.

REQ-MSG-2: The system shall support group messaging over a Group's Conversation, visible to all current GroupMembers.

REQ-MSG-3: The system shall track read/unread status per message.

REQ-MSG-4: The system shall allow a user to End Chat on a direct conversation, which shall set the Connection's status to ended with ended_reason = manual_end and make the conversation inaccessible for further messaging, without affecting feed visibility (distinct from Block, REQ-MATCH-8).

REQ-MSG-5: The system shall deliver new messages to an open conversation via client-side polling.

4.6 AI Translation (Translation Agent)

4.6.1 Description and Priority

Uses the Translation Agent to translate messages on demand or automatically into a recipient's preferred language, degrading gracefully when the underlying API call fails. Priority: Medium.

4.6.2 Stimulus/Response Sequences

- Recipient has auto_translate enabled → a message arrives in a language other than their preferred language → the Translation Agent calls the Google Gemini API (secondary key) and stores the translated text in a new MessageTranslation row.
- If the Gemini call fails, the system stores the original, untranslated text as a placeholder in translated_text and sets is_fallback = true, so the failure is not visible to the user in the moment; the next time that translation is viewed, the system detects is_fallback = true and retries the call rather than trusting the stored placeholder indefinitely.
- User manually requests translation of a specific message not otherwise auto-translated.

4.6.3 Functional Requirements

REQ-TRANS-1: The Translation Agent shall call the Google Gemini API (secondary key) to translate a message's text into a user's specified target language, on request or automatically when `auto_translate` is enabled.

REQ-TRANS-2: The system shall store each distinct requested translation of a message rather than pre-computing translations into every supported language.

REQ-TRANS-3: If the translation call fails, the system shall store the original message text in `translated_text` and set `MessageTranslation.is_fallback` to true, and shall retry the translation the next time that translation is viewed rather than treating the stored placeholder as final.

REQ-TRANS-4: The system shall clearly indicate to the user when displayed text is a translation rather than the original message.

4.7 Notifications

4.7.1 Description and Priority

Informs a student of relevant activity: new matches, messages, mission progress, and system announcements. Priority: Medium.

4.7.2 Stimulus/Response Sequences

- A triggering event occurs (match request received, new message, mission milestone reached, badge earned, nudge generated, system update) → a Notification is created for the affected user → it appears in their Notifications list, optionally with a call-to-action.

4.7.3 Functional Requirements

REQ-NOTIF-1: The system shall generate a Notification for each of the following event types: match, group, message, badge, mission, and system.

REQ-NOTIF-2: The system shall allow a Notification to optionally carry a call-to-action label and link.

REQ-NOTIF-3: The system shall track read/unread status per Notification.

4.8 Missions & Badges

4.8.1 Description and Priority

Gamifies engagement through a tiered, fully deterministic badge system: users complete daily/weekly missions to unlock Bronze, Silver, and Gold tiers of each badge, tracked automatically as they use the platform. Personalized progress nudges are a related but separate capability, described under the Nudge Agent (Section 4.10). Priority: Low.

4.8.2 Stimulus/Response Sequences

- User performs a qualifying action (e.g., sends their first message, joins a group) → the relevant UserMissionProgress and UserEngagement counters update → on reaching a badge tier's threshold, that tier's Badge is awarded via deterministic threshold-checking logic (never AI-driven) and a Notification is generated.

4.8.3 Functional Requirements

REQ-BADGE-1: The system shall track engagement counters per user (messages sent, countries connected, groups joined, translations used, comments made, likes given, conversation count) via UserEngagement.

REQ-BADGE-2: The system shall define missions with a frequency (daily or weekly) and a numeric target, and shall track each user's progress toward the current period's target.

REQ-BADGE-3: The system shall define each badge concept (e.g., "Social Butterfly") as three tiered Badge records — Bronze, Silver, and Gold — sharing a common badge_group value and distinguished by a tier field, each with its own increasing threshold on the same metric.

REQ-BADGE-4: The system shall automatically award the appropriate Badge tier to a user, using deterministic (non-AI) threshold comparison, when their tracked metric reaches or exceeds that tier's threshold, and shall not award the same badge tier twice to the same user.

REQ-BADGE-5: The system shall notify a user when they complete a mission or earn a badge tier.

REQ-BADGE-6: Exact threshold values for each badge tier are configuration data (Badge.threshold), administrator-editable via Django admin, and are intentionally not fixed as part of this specification; representative sample data is provided non-normatively in Appendix B.7.

4.9 AI Safety & Moderation (Safety Agent)

4.9.1 Description and Priority

Uses the Safety Agent to synchronously screen new chat messages, posts, and comments for potentially harmful content before they become visible to anyone, calling the Google Gemini API directly with no third-party moderation API involved. Also provides the manual user-reporting and admin-review workflow. Priority: High.

4.9.2 Stimulus/Response Sequences

- User sends a chat message, creates a post, or writes a comment → the system calls the Safety Agent (Google Gemini API, primary key) and waits for its response before the content becomes visible to anyone → based on severity and category, the content is allowed through, auto-blocked, or queued for human review, and a SafetyFlag record is created describing the finding.
- If the Gemini call fails, times out, or the API quota is exceeded, the system fails open: the content is still sent/posted normally, and the sender/poster sees a brief on-screen notice (e.g., “safety agent is napping”) rather than being blocked.
- A SafetyFlag may optionally be linked to an existing user-filed Report if it corroborates one, and can be independently marked open, reviewed, or dismissed by an administrator — distinct from that Report's own status.
- User reports another user's content or behavior → a Report is created → an administrator reviews it via Django admin → the administrator records a resolution and, if warranted, takes action against the reported user.

4.9.3 Functional Requirements

REQ-SAFE-1: The Safety Agent shall call the Google Gemini API (primary key) directly — with no third-party moderation API — to evaluate new chat messages, posts, and comments for potentially harmful or inappropriate content.

REQ-SAFE-2: The safety check shall be synchronous: a message, post, or comment shall not become visible to other users until the Safety Agent's check completes, or its fallback (REQ-SAFE-3) is triggered.

REQ-SAFE-3: If the Safety Agent's API call fails, times out, or exceeds quota, the system shall fail open — the content shall still be delivered/posted normally — and shall display a brief, non-blocking notice to the sender indicating the safety check could not complete.

REQ-SAFE-4: The system shall record each Safety Agent finding as a SafetyFlag referencing the originating message, post, or comment (exactly one of which shall be set), the flagged user, its severity, category, an AI-generated reasoning summary, and its own status (open/reviewed/dismissed) independent of any linked Report's status.

REQ-SAFE-5: The system shall allow any user to submit a Report against another user, optionally attaching the Conversation in which the incident occurred, with a reason selected from a fixed set of categories.

REQ-SAFE-6: The system shall track each Report's status as pending, reviewed, dismissed, or action_taken, along with which administrator reviewed it, when, and any resolution notes.

REQ-SAFE-7: Action taken as a result of a Report or a reviewed SafetyFlag (e.g., banning or suspending a user) shall be reflected on the User record via is_banned (permanent ban) and/or suspended_until (temporary suspension, nullable) as appropriate, and, where a Connection is involved, shall end it with ended_reason = report_action.

4.10 Progress Nudges (Nudge Agent)

4.10.1 Description and Priority

Generates and rate-limits personalized notifications about a user's badge and mission progress, scoped specifically to badge_progress and mission_progress nudges — distinct from the deterministic badge-awarding logic in Section 4.8, which it never influences. Priority: Low.

4.10.2 Stimulus/Response Sequences

- Periodically, or when a user's progress toward a badge or mission tier changes meaningfully, the Nudge Agent calls the Google Gemini API (primary key) with the user's current progress data for that badge or mission, and generates a short, personalized nudge message.
- The system records the nudge as a NudgeLog row (referencing exactly one of a Badge or a Mission, never both) together with the progress_pct snapshot at send time, and delivers it as a Notification.
- Before sending another nudge for the same badge or mission, the system checks the progress_pct recorded in the most recent NudgeLog entry for that user/badge (or user/mission) pairing, and only sends a new nudge once progress has moved by a meaningful margin, preventing repeated nudges at the same plateau.

4.10.3 Functional Requirements

REQ-NUDGE-1: The Nudge Agent shall call the Google Gemini API (primary key) to generate personalized nudge notification text based on a user's current badge or mission progress data.

REQ-NUDGE-2: The system shall record each nudge sent as a NudgeLog entry referencing exactly one of a Badge or a Mission (never both), the nudge_type (badge_progress or mission_progress), the progress_pct at time of sending, the generated message text, and the send timestamp.

REQ-NUDGE-3: The system shall rate-limit nudges per user per badge/mission using the progress_pct recorded in the most recent NudgeLog entry for that pairing, so a user is not repeatedly nudged without meaningful progress.

REQ-NUDGE-4: Nudge Agent output shall never influence or bypass the deterministic badge-awarding logic specified in REQ-BADGE-4.

4.11 Platform Assistant (Platform Assistant Agent)

4.11.1 Description and Priority

Provides a conversational, platform-scoped help assistant that explains how features work and helps a user find their way around the platform. It is explicitly not a general-purpose writing or content-generation tool. Priority: Low.

4.11.2 Stimulus/Response Sequences

- User opens the Platform Assistant → starts or continues an AssistantThread → sends a question about how to use the platform → the Assistant calls the Google Gemini API (secondary key), scoped to platform guidance, and responds with an AssistantMessage.
- If the Gemini call fails, the system degrades gracefully with a non-blocking on-screen notice, consistent with the fallback pattern used by the other Gemini-backed agents (Section 2.5).

4.11.3 Functional Requirements

REQ-ASSIST-1: The system shall allow a verified user to start and continue one or more AssistantThreads, each containing an ordered sequence of AssistantMessages.

REQ-ASSIST-2: The Platform Assistant shall call the Google Gemini API (secondary key), constrained to platform help/navigation guidance, and shall not be used to generate general-purpose written content (e.g., essays) on the user's behalf.

REQ-ASSIST-3: If the Gemini call fails, the system shall degrade gracefully with a non-blocking on-screen notice rather than blocking the user's ability to continue using the rest of the platform.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Standard page loads (feed, chat inbox, profile) should complete within 2 seconds under normal load on a typical university network connection.
- Sending a message should reflect in the sender's own conversation view within 1 second, independent of Safety Agent processing time (see below).
- The matching pool query should exclude blocked/ineligible users efficiently even as the user base grows, avoiding a full-table scan per request.
- Because the Safety Agent's check is synchronous, sending a chat message, post, or comment carries a brief added latency while the check completes, bounded by the system's request timeout and single retry policy, before the system fails open per REQ-SAFE-3. This is an accepted, deliberate trade-off that favors pre-publication moderation over instant delivery, and is not specified here as a hard numeric limit.
- Every Gemini-backed agent call should degrade gracefully per its fallback strategy defined in Section 2.5, rather than blocking core page rendering or core platform functionality.

5.2 Safety Requirements

- The Safety Agent's automated SafetyFlags shall never be the sole basis for permanently banning a user; flagged content shall route to human review before irreversible account action is taken, given this is a student project without a mature appeals process.
- A SafetyFlag's own status (open/reviewed/dismissed) shall be tracked independently of any linked Report's status, so a flag can be dismissed without ever becoming a formal Report.
- Soft-delete (is_deleted, deleted_at) shall be used for account removal rather than hard deletion, preserving data integrity for any content that references the user.
- Badge-awarding logic (REQ-BADGE-4) shall remain fully deterministic and shall not depend on any AI/LLM output, to prevent incorrect or inconsistent badge awards.

5.3 Security Requirements

- All authentication shall be delegated to Google OAuth; the system shall not store user passwords.
- Gender data shall be restricted to internal matching logic and shall never be exposed via any public-facing API or page.
- Administrator actions performed in Django admin shall be logged automatically (Django's built-in LogEntry), providing an audit trail without additional custom code.
- All communication between client and server, and all calls to the Google Gemini API, shall occur over HTTPS.

5.4 Software Quality Attributes

- Maintainability: the codebase shall be organized into 7 Django apps by domain (accounts, social, matching, chat, engagement, moderation, ai_agents) so each feature area can be understood and modified independently.
- Extensibility: Verification, Matching, Translation, Safety, Nudge, and Platform Assistant logic shall sit behind stable interfaces so the underlying AI provider can be changed without affecting calling code or this specification.

- Resilience: every Gemini-backed agent shall have a defined, agent-appropriate fallback behavior (Section 2.5), so that a single agent's API failure degrades only that feature and never the platform as a whole.
- Usability: the interface shall follow a single consistent design system (color palette, typography, iconography) across all pages.

5.5 Business Rules

- A user may hold at most 5 interests at a time.
- A Block always ends any active Connection between the two users involved; an End Chat never affects feed/post visibility.
- Group joining never requires a MatchRequest or an accept/decline step.
- Exactly one of a Conversation's connection_id or group_id fields shall be set, never both and never neither.
- Exactly one of a SafetyFlag's message, post, or comment fields shall be set.
- Exactly one of a NudgeLog's badge or mission fields shall be set, never both.
- Account suspension is represented via both is_banned (permanent) and suspended_until (temporary, nullable) fields on User, applied depending on the severity of the moderation action taken.
- Each badge concept shall be represented as three tiers (Bronze, Silver, Gold) rather than a single flat badge, so that a highly active user does not exhaust available badges.

6. Other Requirements

No legal, licensing, or regulatory requirements beyond standard data-privacy handling of student email addresses and profile data have been identified at this stage. Reuse requirements: the system shall reuse Django's built-in admin interface rather than building a custom administrative UI, per the project's implementation constraints.

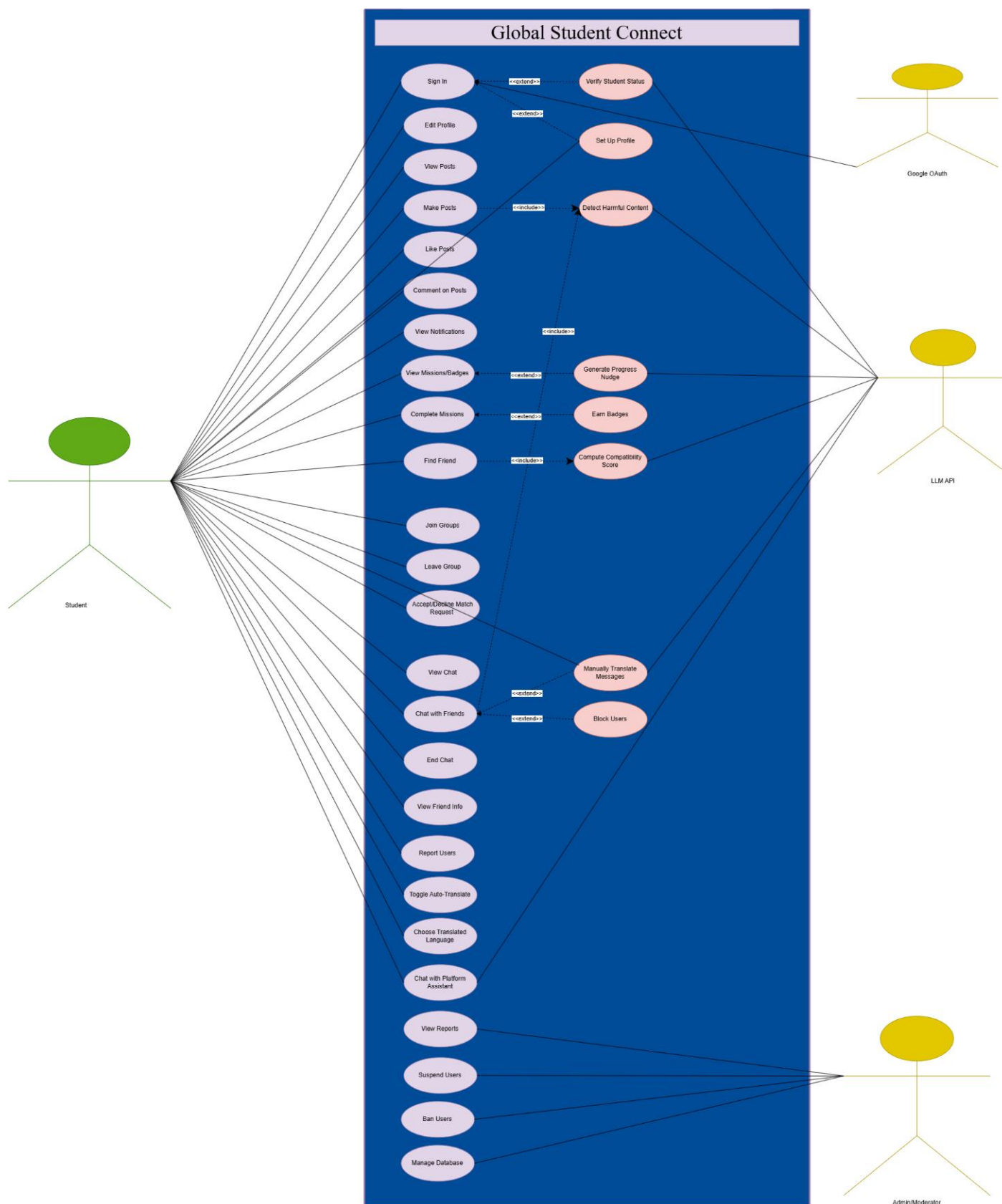
Appendix A: Glossary

- **Agent** — one of six named logical components (Verification, Matching, Translation, Safety, Nudge, Platform Assistant) responsible for a specific area of AI-assisted behavior, each implemented as a genuine call to the Google Gemini API with a defined, agent-appropriate fallback for when that call fails. Badge/mission threshold-tracking and awarding logic remains fully deterministic and is not itself an Agent.
- **Connection** — a persistent link between two users created when a MatchRequest is accepted; carries a status (active/ended) and, when ended, a reason.
- **Block** — a directional record that a user has blocked another; enforced with symmetric visibility effects across the platform.
- **End Chat** — the action of ending a direct Connection's messaging without affecting feed/post visibility, distinct from Block.
- **Moment** — a short text post shared to the Home feed.
- **Mission** — a recurring (daily/weekly) engagement goal tied to a numeric target and optionally linked to a Badge.
- **Badge** — an achievement unlocked once a user's tracked engagement metric reaches a defined threshold.
- **Tier** — one of three levels (Bronze, Silver, Gold) within a single badge concept, each with its own increasing threshold on the same metric.
- **Badge Group** — a shared identifier linking the Bronze, Silver, and Gold Badge records that belong to the same badge concept (e.g., "social_butterfly").
- **Verified** — a User whose email domain has matched a recognized academic suffix pattern (is_verified = true); independent of whether that domain's University.name was generated by the Verification Agent or its deterministic fallback.
- **SafetyFlag** — a record of an AI-detected potential issue in a message, post, or comment, independent of and optionally linked to a human-filed Report; carries its own severity, category, AI-generated reasoning, and status.
- **Fail open** — the Safety Agent's fallback behavior: on API failure, content is still delivered/posted rather than blocked, with a non-blocking notice shown to the user.
- **is_fallback** — a flag on MessageTranslation indicating the stored translated_text is a placeholder (the original, untranslated text) resulting from a failed Translation Agent call, to be retried on next view rather than trusted as final.
- **NudgeLog** — a record of a single Nudge Agent notification sent to a user about their progress on exactly one Badge or Mission, including the progress_pct snapshot used for rate-limiting future nudges.
- **AssistantThread / AssistantMessage** — the conversation and per-turn message records for a user's interaction with the Platform Assistant.

Appendix B: Analysis Models

This appendix contains the UML analysis models supporting this SRS: the Use Case Diagram, a detailed Use Case Description for the system's most important use case, two Sequence Diagrams and one Communication Diagram for the three most important use cases, an Activity Diagram for the use case whose decision-making logic is best understood as a workflow, and the detailed Class Diagram.

B.1 Use Case Diagram

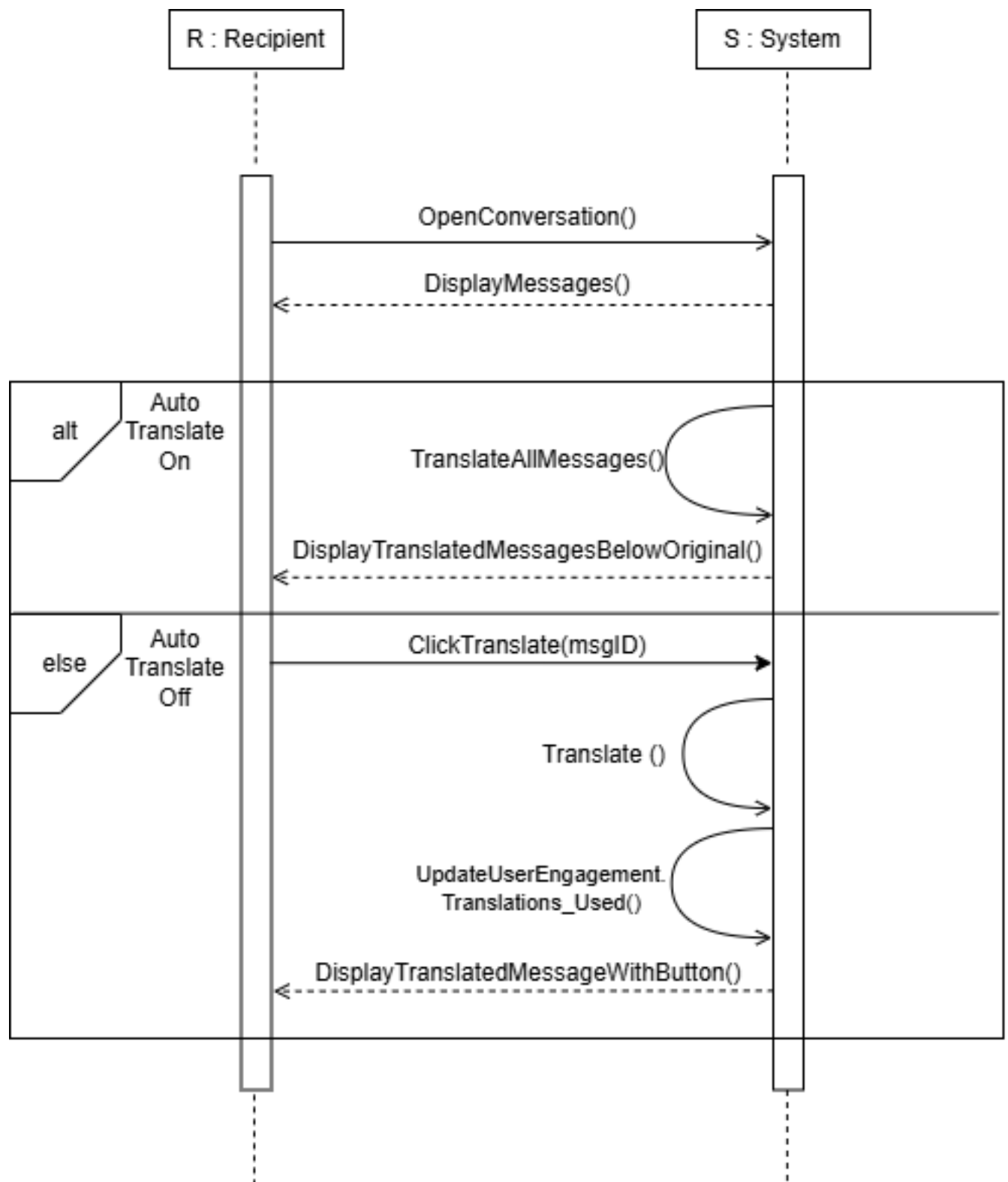


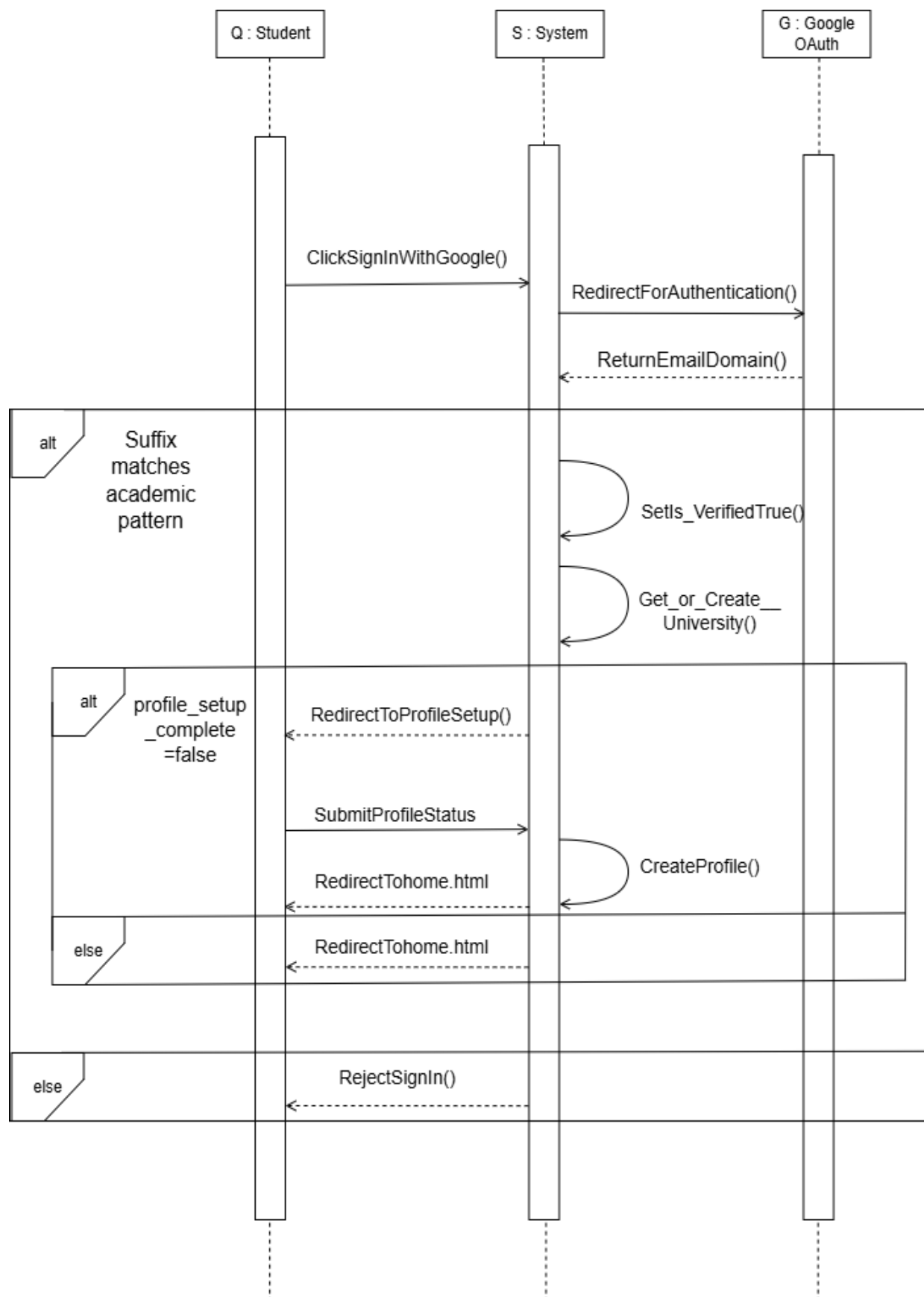
B.2 Detailed Use Case Description: Find a Friend

This is the system's most important use case: it is the primary driver of new Connections, exercises the Matching Agent end-to-end (including its deterministic fallback), and touches five of the seven Django apps (accounts, matching, chat, engagement, ai_agents).

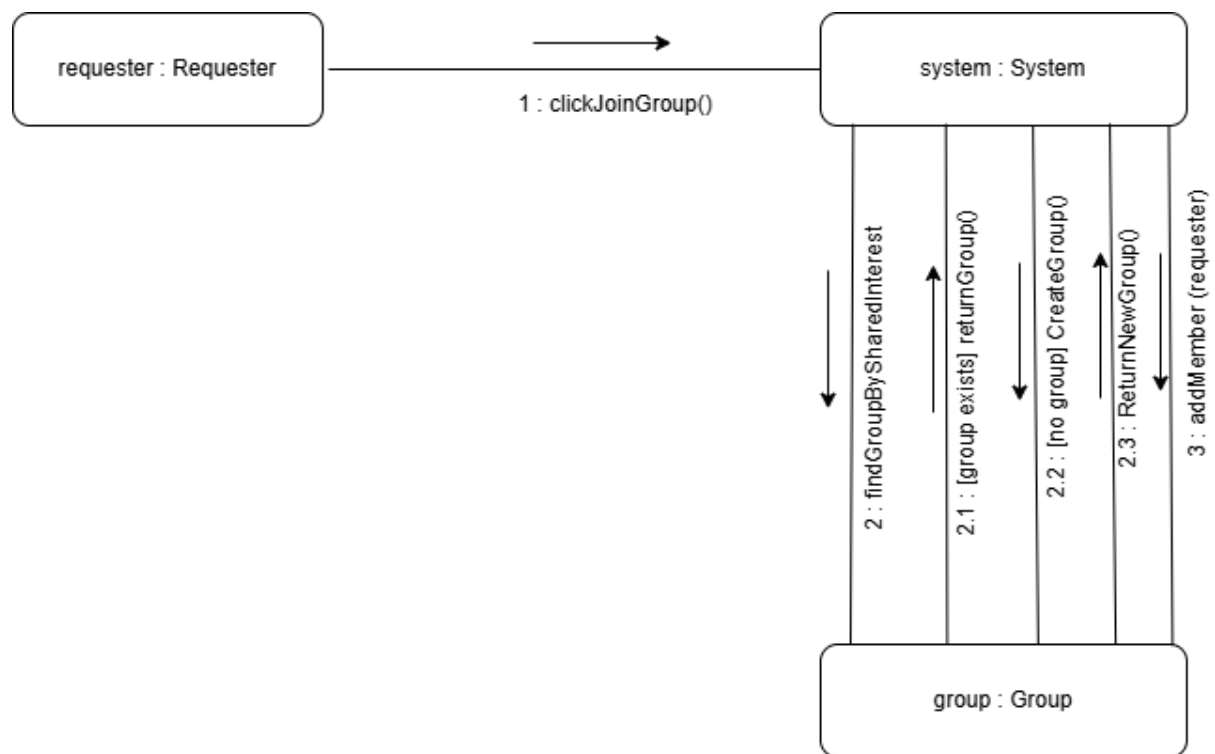
Field	Description
Use Case Name	Find a Friend
Primary Actor	Student (Requester)
Secondary Actor	Student (Recipient); Matching Agent (system-internal)
Preconditions	The Requester is signed in and verified (is_verified = true), has completed profile setup, and has no MatchRequest they sent that is still pending.
Trigger	The Requester selects “Find Friend” on the Connect page.
Main Success Scenario	1. The Requester selects Find Friend. 2. The system fetches the candidate pool (verified students excluding the Requester, blocked users, and users with an existing active Connection to the Requester). 3. The Matching Agent calls the Google Gemini API to compute a compatibility score for each candidate and selects the single highest-scoring candidate. 4. The system creates a MatchRequest (status = pending) from the Requester to the selected candidate and shows the Requester a waiting screen. 5. The Recipient sees a handshake-icon indicator on their Connect page and receives a Notification. 6. The Recipient opens the incoming-requests modal and views the request card. 7. The Recipient accepts the request. 8. The system updates the MatchRequest to accepted, creates a Connection (status = active) and its associated Conversation, and updates engagement tracking. 9. The Requester's waiting screen resolves, revealing the matched student and the compatibility score as a multicolored bar with its percentage. 10. The Requester clicks Start Chatting and is navigated to the new conversation; the Recipient's conversation is added to their inbox and they are navigated there automatically.
Extensions (Alternate Flows)	2a. No candidates are available: the system shows “No students available”; the Requester may explore other features. 3a. The Gemini call fails: the system falls back to a deterministic in-code scoring algorithm and continues at step 4 with no visible error. 4a. The Requester cancels the pending request: the MatchRequest is set to cancelled; if no further requests are queued for the Recipient, they see an “all caught up” state. 7a. The Recipient declines instead: the MatchRequest is set to declined and removed from the queue; if more requests are queued for the Recipient, the next one is shown; otherwise the Requester sees “No match found” and may try again or join a group instead.
Postconditions (Success)	A Connection and Conversation exist between the Requester and Recipient; both users can now message each other; engagement counters are updated for both.
Postconditions (Failure)	No Connection is created; the MatchRequest is left in a terminal state (cancelled or declined); the Requester may attempt Find Friend again.
Priority	High

B.3 Sequence Diagrams





B.4 Communication Diagram



B.5 Activity Diagram: Find a Friend

The workflow below spans two pages due to its length. Off-page connectors A and B mark where the flow continues from the fork/join bar at the bottom of page 1 to the top of page 2.

Figure B.5a — Find a Friend Activity Diagram (page 1 of 2)

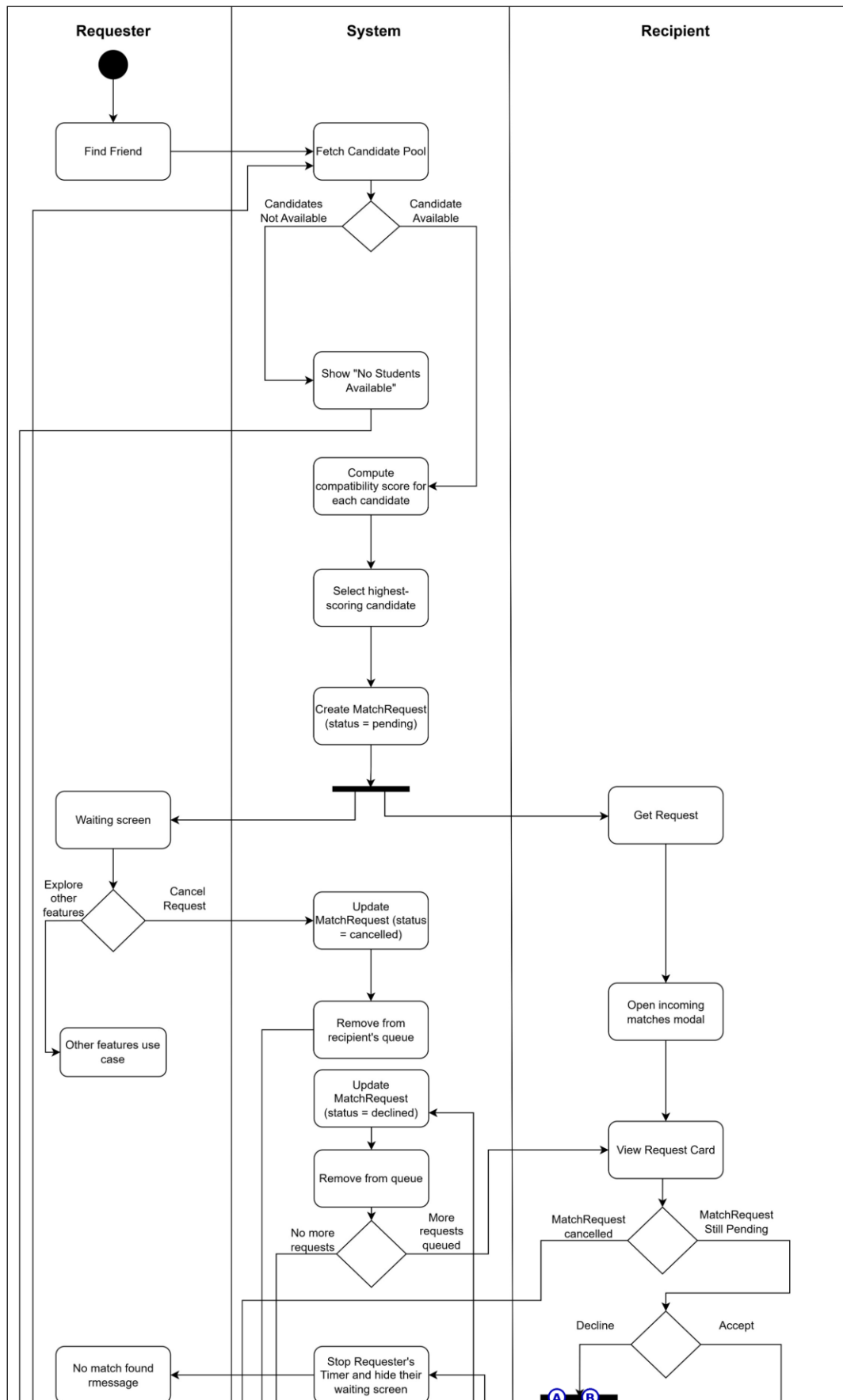
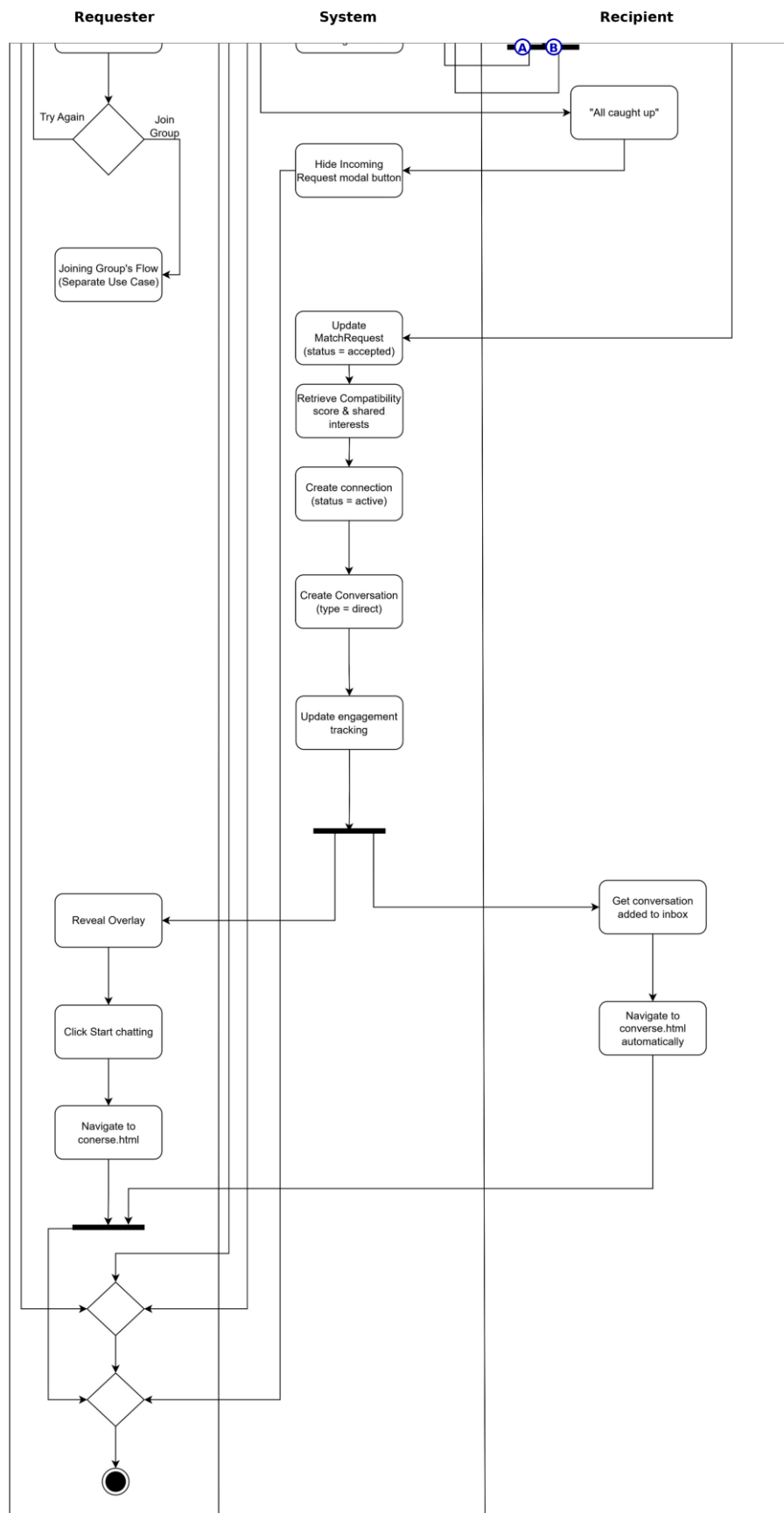
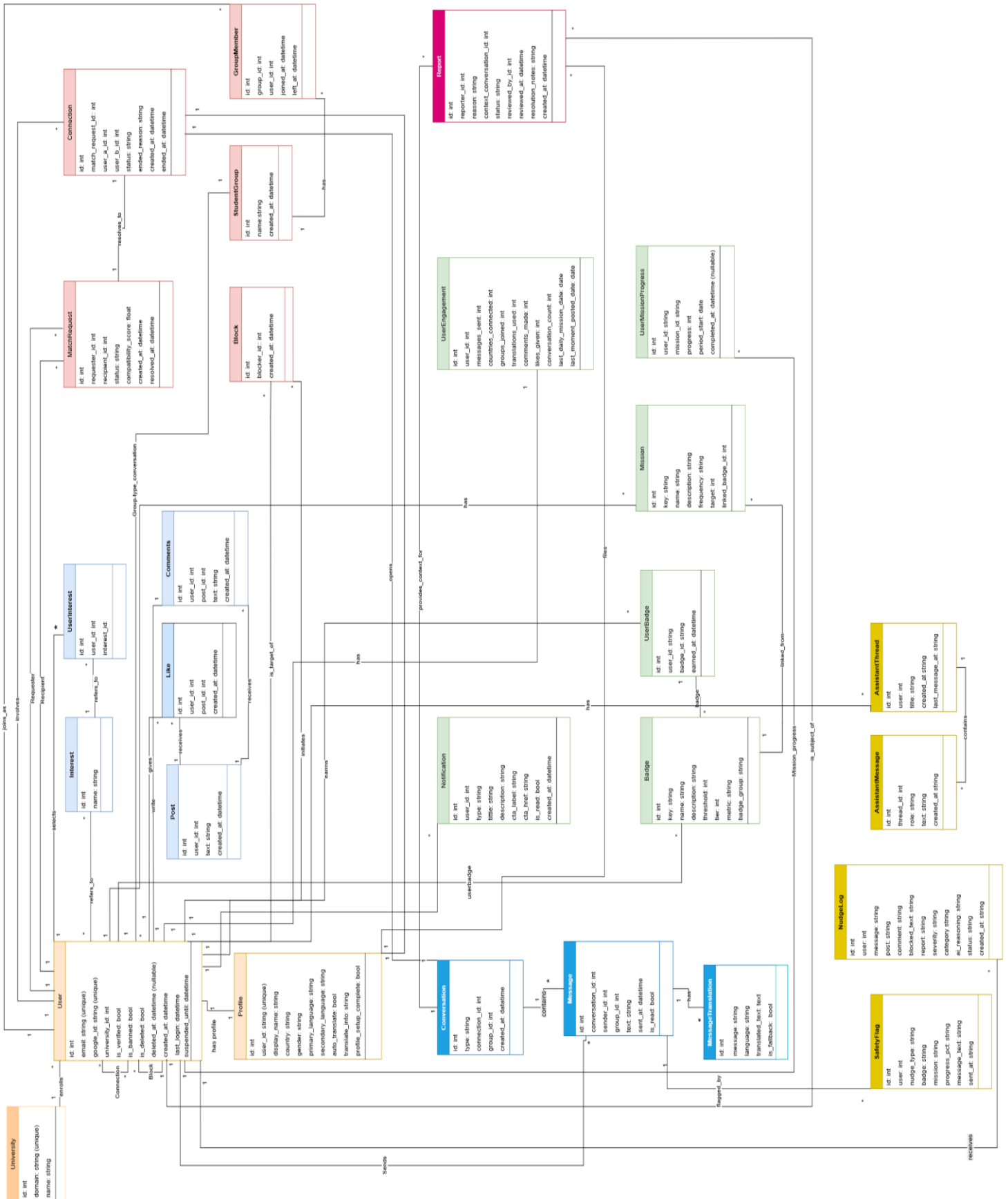


Figure B.5b — Find a Friend Activity Diagram (page 2 of 2, continued via connectors A, B)



B.6 Class Diagram



Appendix C: To Be Determined List

- TBD-1: Whether a supplementary breathing-exercise/relaxation feature is added to the platform. This is a stretch goal the team will include if development time allows; it is not yet scoped (screens, trigger points, or data model).
- TBD-2: The exact on-screen notice wording and placement for the Translation and Platform Assistant agents' graceful-fallback states (REQ-TRANS-3, REQ-ASSIST-3). The Safety Agent's equivalent notice is already defined ("safety agent is napping"); Translation and Platform Assistant are not yet implemented as of this revision.
- TBD-3: Exact CSS styling values (colors, spacing, component-level visual detail) may still change from what is described in Section 3.1; the design system listed there reflects the current prototype and is subject to refinement.